반복은 AI가 먼저, 판단은 사람이

반복되는 장애 분석과 사내 업무 처리를 AI가 먼저 맡도록 바꾸고, 사람은 검토·승인과 핵심 개발 업무에 집중하는 구조를 만듦.



두 사례에서 바꾼 일의 흐름. 첫 줄은 사례 1, 둘째·셋째 줄은 사례 2.

사례 1 · 실제 운영 중

소수 인력의 장애 대응 병목을 AI 에이전트로 해결

문제	에러마다 로그 확인부터 수정·배포까지 사람이 직접 처리해, 대응 속도가 담당자의 시간에 좌우됨
AI 활용	AI가 1차 리뷰와 테스트를 통과한 수정 PR을 만들고, 사람은 검토·승인
결과물	모든 에러에 1차 리뷰, 이슈의 약 70%는 수정 PR까지, PR의 약 80%는 추가 수정 없이 머지
담당	기획·설계·구현·운영 전 과정

Fluent Bit · FastAPI · LangChain · Gemini · GitHub · CI/CD

사례 2 · 파트원 공동 사용 중

흩어진 사내 업무 규칙을 문서화해 AI가 문의 응답과 반복 작업을 처리하게 함

문제	절차는 매번 서무에게 확인하고, 정해진 포맷의 반복 작업은 손으로 처리함
AI 활용	사내 정보와 작업 규칙을 md 문서로 정리하고, 회사가 허용한 Claude가 참조해 답과 산출물을 만듦
결과물	비용 처리는 AI에게 물어 바로 답을 얻고, 방화벽 CSV는 AI가 만든 뒤 담당자가 확인해 제출

Obsidian · Markdown · Claude

사례 1 **장애 대응 AI 에이전트** 문제 2 · AI 활용 3 · 처리 순서와 결과물 예시 4 · 성과와 안전장치 5

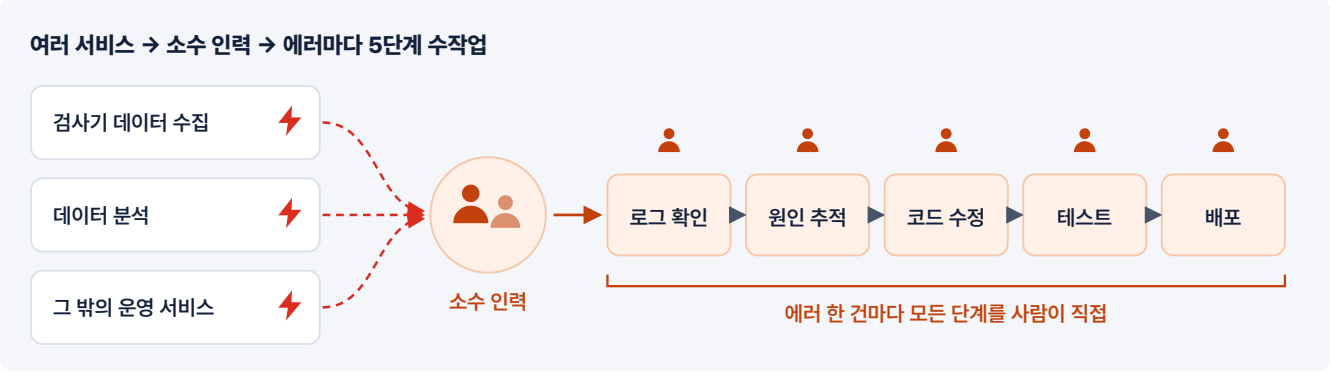
사례 2 **업무 규칙 문서화** 문제와 AI 활용 6 · 방화벽 신청 자동화와 두 사례 정리 7

그림 읽는 법 사람이 하는 일 AI가 하는 일 시스템·도구

문제

에러 대응 속도가 담당자의 시간에 묶여 있었음

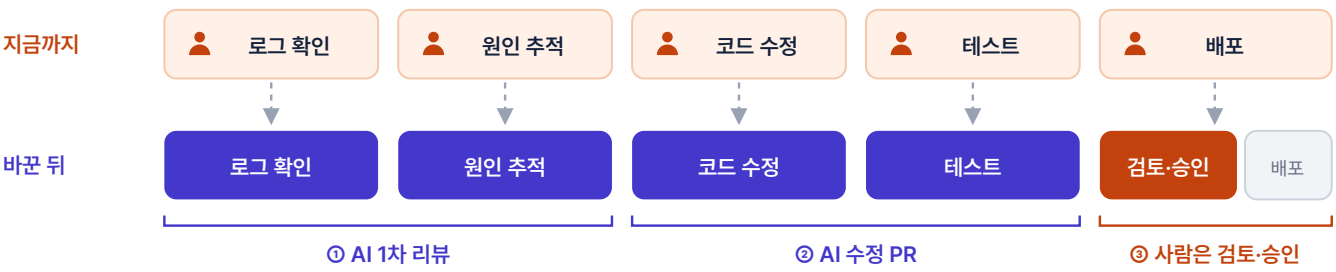
휴대폰 검사기 데이터 수집 파이프라인과 데이터 분석 서비스는 소수 인력이 함께 운영함. 에러 하나마다 로그를 직접 확인하고, 원인을 추적해 수정·테스트·배포하는 과정을 모두 사람이 해야 했음. 담당자가 다른 일을 하고 있거나 퇴근한 뒤에 에러가 나면 누군가 확인하기 전까지 손을 쓸 수 없었고, 대응 속도는 담당자가 언제 볼 수 있느냐에 달려 있었음.





대응이 시작되는 시점은 에러가 난 순간이 아니라, 담당자가 볼 수 있게 된 순간이었음. 개념도

> **병목은 에러를 발견하는 것이 아니라, 분석하고 고칠 사람의 시간이었음.** 그래서 사람이 하던 단계 자체를 AI에게 넘기기로 함.

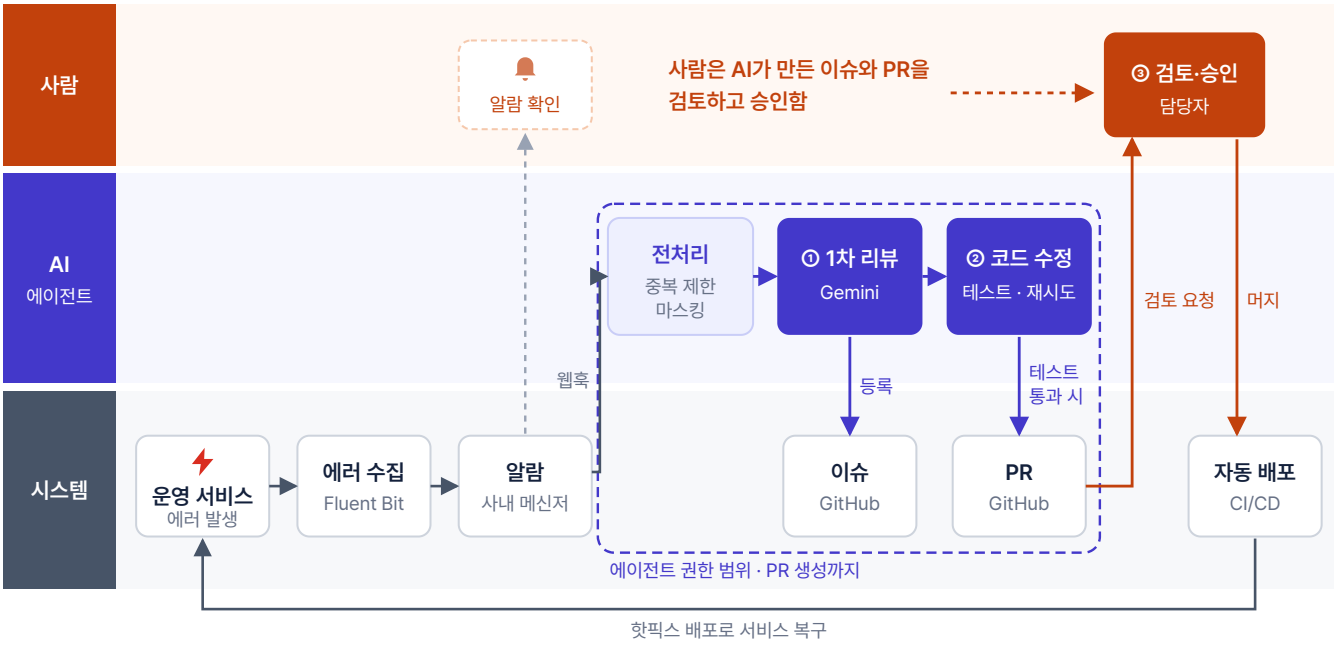


같은 다섯 단계에서 맡는 주체를 바꿈. 사람은 AI 결과를 검토하고 승인함.

AI 활용

분석·수정·검증은 AI 에이전트가, 검토·승인은 사람이

사람이 하던 분석 → 수정 → 검증을 AI 에이전트에게 넘기되, AI가 자동으로 처리할 범위와 사람이 승인할 지점을 나눠 파이프라인을 설계·구축함.



누가 무엇을 하는지로 나눈 전체 흐름. 점선 안이 에이전트 권한 범위이고, 머지와 배포는 사람의 승인을 거침.

1 AI 1차 리뷰

사람 대신 분석

Fluent Bit으로 모은 에러가 사내 메신저 알람과 함께 에이전트 서버(FastAPI, LangChain)로 전달되면, 외부 LLM(Gemini)이 마스킹된 로그를 분석해 원인·영향 범위·수정 방향·우선순위를 GitHub 이슈에 작성함.

2 AI 수정 PR

사람 대신 수정·검증

에이전트가 에러의 서비스 이름을 서비스-저장소 매핑 문서와 대조해 대상 저장소를 찾아 클론하고, 관련 코드를 수정한 뒤 테스트를 직접 실행함. 통과한 수정만 PR로 올리고, 실패하면 다시 수정을 시도한 뒤 끝내 실패하면 PR 없이 1차 리뷰만 남김.

3 사람은 검토·승인

검토하고 승인

담당자가 이슈와 PR을 검토해 머지하면 CI/CD가 핫픽스를 배포함. AI가 만든 PR도 사람의 승인 없이는 운영에 반영되지 않음.

에러 수집 **Fluent Bit**	알람 · 트리거 **사내 메신저 · 웹훅**	에이전트 서버 **FastAPI · LangChain**	분석 · 수정 **Gemini · 회사 승인 계약**
이슈 · PR **GitHub**	배포 **CI/CD**	에이전트 권한 **PR 생성까지**	대상 저장소 찾기 **서비스-저장소 매핑 문서**

본인 역할 기획, 에이전트 설계, FastAPI·LangChain 구현, GitHub·CI 연동, 운영까지 전 과정을 직접 맡음.

AI 활용 상세

에러 한 건이 검증된 PR이 되기까지

에이전트는 알람을 받은 순간부터 정해진 단계를 따름. 테스트를 통과한 수정만 PR로 올리고, PR 생성에 실패해도 원인·영향 범위·수정 방향을 담은 1차 리뷰는 남아 담당자는 AI의 선행 분석을 보고 대응을 시작함.



에이전트의 처리 순서

비식별 재구성 예시 · 실제 화면 아님

사내 메신저 · 에러 알람 02:13
[ERROR] 검사 결과 수집 서비스
`KeyError: 'defect_code'`

↓ 웹훅 → 에이전트가 1차 리뷰 작성

Open ai-agent가 작성한 이슈

[AI 1차 리뷰] 검사 결과 수집 서비스 · 필수 필드 누락 예외

ai-review 우선순위 높음

원인 추정	검사 결과 응답에서 선택 필드 `defect_code`가 비어 오는 경우를 처리하지 않음. 위치는 `ResultHandler` 87행
영향 범위	이 필드를 비워 보내는 검사기의 결과 저장이 실패함. 나머지 검사기는 정상
수정 방향	빈 값이면 기본값으로 처리하고, 해당 케이스의 테스트를 추가
우선순위	높음 (검사 결과 저장 누락) · 담당: 결과 수집 서비스

↓ 코드 수정 → 테스트 통과

Open PR ai-agent → main · 위 이슈와 연결

fix: 빈 defect_code 기본값 처리 및 테스트 추가

```
- code = payload["defect_code"]
+ code = payload.get("defect_code") or DEFAULT_CODE
```

✓ 테스트 통과 · 통과한 뒤에만 PR이 열림

담당자 검토 대기 · 승인(머지)하면 배포

에이전트가 받는 것

- 마스킹된 에러 로그
- 매핑 문서로 찾은 서비스 저장소
- 테스트 실행 결과

→

AI가 하는 일 · Gemini + LangChain

- 원인·영향 범위·수정 방향·우선순위 판단
- 코드 수정과 테스트 실행, 실패 시 재시도

→

담당자가 받는 것

- GitHub 이슈의 1차 리뷰
- 테스트를 통과한 PR
- 통과 못 하면 리뷰만

성과 ● 실제 운영 중 · 배포는 담당자 승인 후

모든 에러에 1차 리뷰, 약 70%는 수정 PR까지

1차 리뷰 자동 작성

100%

중복을 묶은 뒤 모든 에러가 GitHub 이슈와 1차 리뷰로 자동 작성됨

기준: 수집된 에러

→

테스트를 통과한 수정 PR

약 70%

수정 PR까지 생성됨. 나머지는 PR 없이 1차 리뷰만 남김

기준: 자동 생성된 이슈

→

추가 수정 없이 머지

약 80%

담당자가 코드를 고치지 않고 그대로 머지함. 나머지는 보완하거나 반려

기준: AI가 만든 PR

운영 중 집계한 대략값이며, 수치마다 기준(분모)이 다름.



같은 밤사이 에러가 도입 전과 후에 처리되는 방식. 수정 PR은 이슈의 약 70%에서 준비됨.

개념도 · 구간 길이는 실측값 아님

- ✓ **로그 분석부터 승인 후 배포까지 전체 흐름을 실제 운영 중임.**
- ✓ **AI가 PR까지 만든 에러는 유형이 다양함.** 비즈니스 로직, 데이터 파싱, 설정·배포 환경 오류 등.
- ✓ **밤사이 난 에러도 출근하면 원인 분석이 끝나 있음.** 상당수는 수정 PR까지 준비되어 있음.
- ✓ **담당자의 일이 검토·승인 중심으로 바뀜.** 장애 때마다 개발을 멈추던 부담이 줄어 개발 업무에 집중할 수 있게 됨.

안전장치 정보 보호 · 에이전트 통제 · 배포 통제의 세 겹

정보 보호

외부 LLM은 회사가 승인한 계약으로 사용하고, 민감 정보는 마스킹한 뒤 보냄.

LLM 호출 전

에이전트 통제

같은 에러는 시간 윈도우로 묶고 재시도 횟수를 제한해 AI 호출 폭증을 막음. 권한은 PR 생성까지.

에이전트 실행 중

배포 통제

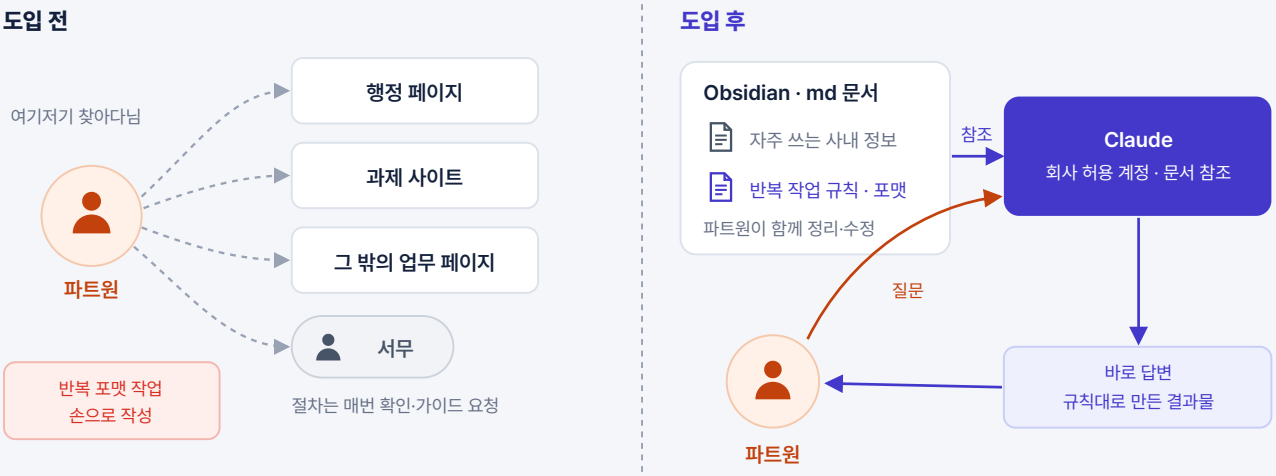
테스트 통과는 PR을 올리는 최소 조건으로만 쓰고, 머지와 배포는 담당자가 최종 판단함.

PR 이후

사례 1 정리 | **사람이 있어야 시작되던 장애 대응이, 사람이 보기 전에 AI가 먼저 진행해 두는 구조로 바뀜.**

문제 · AI 활용 ● 파트원 공동 사용 중

흩어진 업무 규칙을 문서로 모으고, 처리는 AI에게



정보를 찾아다니고 묻던 구조에서, 정리된 문서를 AI가 대신 읽고 답하는 구조로.

문제

행정·과제 등 사내 업무 정보가 여러 사이트에 흩어져 있어, 비용 처리 같은 절차는 매번 서무에게 확인하고 가이드를 받아야 했음. 정해진 포맷의 반복 작업도 손으로 처리함. 예를 들어 방화벽 신청은 출발지·도착지·포트를 지정된 CSV 포맷으로 써야 하는데, 출발지 4개와 도착지 3개면 12개 조합을 일일이 작성하고 이를 법인별로 나눠 올려야 했음.

AI 활용

자주 쓰는 사내 정보와 반복 작업의 규칙·결과 포맷을 Obsidian에서 md 문서로 정리하고, 회사가 허용한 Claude가 이 문서를 참조해 답하고 산출물을 만들도록 함. 작업마다 스크립트를 만드는 대신 규칙 문서만 추가하면 되므로, 규칙이 바뀌거나 새 작업이 생겨도 문서만 고치면 됨. 방화벽 신청은 조합 방식, CSV 컬럼 형식, 법인별 작성 방식을 규칙 파일로 정리함.

문서 구성

- 업무 지식 베이스
  - 행정
    - 비용처리-절차.md
  - 과제
    - 과제-사이트-안내.md
  - 반복업무
    - **방화벽신청-규칙.md**

구성 예시

작업마다 스크립트를 만들면
새 작업마다 코드를 짜야 하고, 규칙이 바뀌면 코드를 고쳐 다시 배포해야 함.

규칙 문서 + Claude
새 작업은 md 문서만 추가하고, 규칙이 바뀌면 문서만 고침. 파트원 누구나 고칠 수 있음.

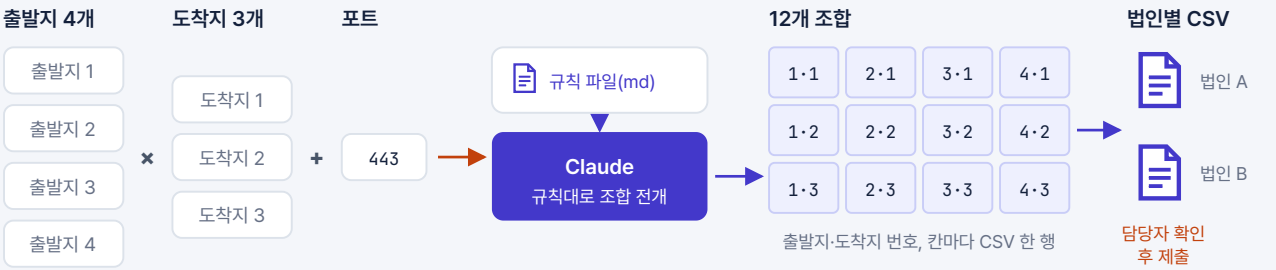
AI를 쓴 이유: 업무 규칙을 코드가 아닌 문서로 관리

결과물

- ✓ **파트원이 함께 쓰는 업무 지식 베이스가 됨.** 규정이 바뀌면 파트원이 함께 문서를 고쳐 최신 상태를 유지함.
- ✓ **비용 처리 방법은 서무에게 묻는 대신 AI에게 물어 바로 답을 얻음.**

결과물

방화벽 신청 CSV는 AI가 만들고, 사람이 확인



사람이 하는 일: 출발지·도착지·포트 입력, 결과 확인 후 제출

규칙 파일 하나로, 조합을 펼치고 법인별로 나누는 일을 AI가 대신함. 예시

방화벽신청-규칙.md 규칙 파일 예시

방화벽 신청 규칙

조합 방식

출발지 × 도착지 × 포트의 모든 조합을 한 행씩 작성

CSV 컬럼 형식

법인, 출발지, 도착지, 포트

법인별 작성 방식

법인마다 CSV 파일을 나눠 작성

질문

출발지 4개, 도착지 3개 주소와 포트 443으로 방화벽 신청 CSV 만들어줘.

```
법인,출발지,도착지,포트
법인A,192.0.2.11,198.51.100.21,443
법인A,192.0.2.11,198.51.100.22,443
법인A,192.0.2.11,198.51.100.23,443
... 12개 조합을 법인별 파일로 나눠 출력
```

출력 예시 · 주소는 문서용 예시 주소

결과물

- ✓ **방화벽 신청은 출발지·도착지·포트를 말로 알려주면** AI가 규칙대로 모든 조합을 펼친 법인별 CSV를 만들고, 담당자가 확인한 뒤 제출함.
- ✓ **서무에게 묻고 CSV를 손으로 채우던 시간을 본 업무에 쓰고 있음.**

두 사례 정리 사람에게 몰리던 일, AI가 먼저 하는 일, 사람에게 남긴 일

	사례 1 · 장애 대응 AI 에이전트	사례 2 · 업무 규칙 문서화
사람에게 몰리던 일	에러마다 로그 확인·원인 추적·수정·테스트·배포	사내 절차 문의, 정해진 포맷의 반복 작성
AI가 먼저 하는 일	**모든 에러의 1차 리뷰, 이슈의 약 70%에서 테스트를 통과한 수정 PR**	**사내 정보 답변, 규칙대로 법인별 CSV 생성**
사람에게 남긴 일	**이슈·PR 검토와 승인, 그리고 개발 업무**	**입력과 결과 확인, 그리고 본 업무**
사용한 AI · 도구	Gemini(회사 승인 계약), LangChain, FastAPI, Fluent Bit, GitHub, CI/CD	Claude(회사 허용), Obsidian, Markdown

두 사례 모두 사람에게 몰리던 반복 업무를 AI가 먼저 처리하는 흐름으로 바꿔, 사람은 검토·판단과 더 중요한 일에 집중하게 만든 경험임.